

GIS Coverage System Explained

A beginner-friendly guide to understanding how RunMap determines street coverage from a single GPS track using PostGIS spatial operations.

Overview: From GPS Track to Coverage

When a single row gets inserted into `runs_raw`, a GIS pipeline processes it to determine which street segments you've covered. This document explains each spatial operation step-by-step.

The Three-Step Pipeline

Step 1: Buffer the Run (Creating a Coverage Zone)

Location: `runmap.buffer_one_run_subdiv(run_uuid)` in `db/04_segment_schema.sql:66-94`

What happens:

1. **Transform coordinates** - Takes your GPS track (a LineString in EPSG:4326) and transforms it to UTM (EPSG:32610)
 - Why? UTM gives us accurate meter-based measurements
 - WGS84 (4326) uses degrees, which vary in distance by latitude
 - UTM (32610) uses meters consistently

2. **Create buffer** - Uses `ST_Buffer()` to create a 10-meter buffer around the track

```
sql ST_Buffer( ST_Transform(geom, 32610), 10.0, -- 10 meters 'endcap=flat join=mitre quad_segs=4' )
```

- This creates a polygon representing "everywhere you could have been within 10m of the GPS track"
- Accounts for GPS accuracy (± 5 -10m typical)
- The buffer compensates for GPS drift and wander

3. **Subdivide** - Breaks the buffer polygon into smaller pieces using `ST_Subdivide(256)`

```
sql ST_Subdivide(buffer_polygon, 256) -- Max 256 vertices per piece
```

- Stored in `runs_buffered_subdiv` table
- **Why?** Smaller geometries = faster spatial index lookups
- Instead of testing "does street intersect huge polygon?" we test "does street intersect any of these small polygons?"

Step 2: Mark Segments as Visited

Location: `runmap.apply_run_to_segments(run_uuid)` in `db/04_segment_schema.sql:97-135`

The clever part - this is where the spatial magic happens!

Streets have been **pre-segmented** into ~5 meter chunks and stored in `block_segments`. Each segment has: -

`segment_id` - Unique identifier - `block_id` - Parent block (connected group of street segments) - `segment_geom` -

LineString geometry (5m piece of street) - `segment_length_m` - Precomputed length in meters

The function executes this spatial query:

```
UPDATE runmap.block_segment_visited bsv
SET visited = TRUE
FROM runmap.block_segments bs
JOIN runmap.runs_buffered_subdiv rb ON rb.run_id = run_uuid
WHERE bsv.segment_id = bs.segment_id
AND bsv.visited = FALSE -- Only update unvisited segments
AND ST_Intersects(bs.segment_geom, rb.geom) -- ★ THE KEY GIS OPERATION
```

SQL

ST_Intersects(segment_geom, buffer_geom) answers:

"Does this 5m piece of street touch the 10m buffer around my run?"

- If YES → segment gets marked `visited = TRUE`
- If NO → segment stays unvisited
- It's a simple boolean flag flip - no geometry storage needed!

How the spatial index makes this fast:

The GIST index on `block_segments.segment_geom` allows PostGIS to: 1. Quickly find candidate segments near the buffer (bounding box check) 2. Then precisely test intersection only for nearby candidates 3. Skip testing thousands of segments far from your run

Step 3: Aggregate Coverage per Block

Still in the same function - After marking segments, it aggregates:

```
INSERT INTO runmap.block_coverage_segments (block_id, visited_length_m, total_length_m)
SELECT
  nv.block_id,
  SUM(nv.segment_length_m) as new_visited_length, -- Add up newly visited lengths
  (SELECT SUM(segment_length_m)
   FROM runmap.block_segments
   WHERE block_id = nv.block_id) as total
FROM newly_visited nv
GROUP BY nv.block_id
ON CONFLICT (block_id) DO UPDATE
SET visited_length_m = block_coverage_segments.visited_length_m + EXCLUDED.visited_length_m
```

SQL

Calculates: - `visited_length_m` = sum of all visited segment lengths in this block - `total_length_m` = total length of all segments in block - `coverage_pct` = (visited / total) × 100 (generated column)

The incremental magic: - First run on a block: `visited_length_m = 15m` - Second run adds 8m more:

`visited_length_m = 15m + 8m = 23m` - Segments marked in first run don't get double-counted (WHERE visited = FALSE)

Why This Design is Brilliant

Traditional Approach (Geometry Union)

```
-- This gets slower with each run and uses tons of memory!  
covered_geom = ST_Union(covered_geom, new_intersection)  
covered_length = ST_Length(covered_geom)
```

SQL

Problems: - `ST_Union` creates increasingly complex geometries - Memory usage grows: $O(\text{total runs} \times \text{streets touched})$ - Processing time increases with each run - 11 runs caused 112GB memory usage and crashes

Segment-Visited Approach

```
-- Constant time! Just binary flags  
visited = TRUE -- Boolean flip, instant
```

SQL

Benefits: - Boolean flags are tiny (1 bit per segment) - Memory usage: $O(\text{segments touched by this run only})$ - Processing time stays constant regardless of total run count - 1000+ runs will process at same speed as first run

Key GIS Concepts Demonstrated

1. Coordinate Reference Systems (CRS)

- EPSG:4326 (WGS84) - Storage format, degrees lat/lon
- EPSG:32610 (UTM Zone 10N) - Calculation format, meters
- EPSG:3857 (Web Mercator) - Display format for web maps

2. `ST_Transform` - Converting between coordinate systems `sql ST_Transform(geom, 32610) -- WGS84 → UTM`

3. `ST_Buffer` - Creating proximity zones around geometries

```
sql ST_Buffer(linestring, 10.0) -- 10m buffer around line
```

4. `ST_Subdivide` - Breaking large geometries into manageable pieces

```
sql ST_Subdivide(polygon, 256) -- Max 256 vertices per piece
```

5. `ST_Intersects` - Testing spatial relationships `sql ST_Intersects(geom_a, geom_b) -- Does A touch B?`

6. `ST_Length` - Measuring linear geometries

```
sql ST_Length(linestring) -- Returns length in CRS units (meters for UTM)
```

7. `GIST Index` - Spatial indexing for fast lookups `sql CREATE INDEX idx_geom ON table USING GIST(geom);`

Visual Flow Diagram

```

GPS Track (runs_raw.geom)
|
├─ LineString in EPSG:4326 (WGS84 lat/lon)
|
↓ ST_Transform(geom, 32610)
|
UTM LineString (meters)
|
↓ ST_Buffer(geom, 10m)
|
10m Polygon Buffer (single large polygon)
|
↓ ST_Subdivide(buffer, 256)
|
Multiple Small Polygons
| (stored in runs_buffered_subdiv)
|
↓ JOIN with block_segments
↓ ST_Intersects(segment_geom, buffer_polygon)
|
Boolean Result: Does street segment touch buffer?
|
├─ YES → UPDATE block_segment_visited SET visited = TRUE
|
↓ GROUP BY block_id
↓ SUM(segment_length_m WHERE visited = TRUE)
|
Coverage Percentage per Block
| (stored in block_coverage_segments)
|
↓ coverage_pct = (visited_length_m / total_length_m) × 100
|
Final Result: "This block is 87% complete"

```

Database Tables Involved

Input Table

- `runmap.runs_raw` - Your GPS track (one row per run)

Intermediate Tables

- `runmap.runs_buffered_subdiv` - Subdivided 10m buffers (many rows per run)
- `runmap.block_segments` - Pre-segmented streets (~5m pieces, one-time computation)

Output Tables

- `runmap.block_segment_visited` - Binary flags (one row per segment, visited = TRUE/FALSE)
- `runmap.block_coverage_segments` - Aggregated coverage (one row per block, with percentage)

Why Segments Are ~5 Meters

Too small (1m segments): - Millions of segments to test - Index overhead outweighs benefits - More rows = more memory

Too large (50m segments): - Coarse granularity - Miss partial coverage details - A 50m segment is either 100% covered or 0% covered

Just right (5m segments): - Fine enough to detect partial coverage - Small enough for accurate percentages - Large enough for efficient indexing - Typical city block = 100-300m = 20-60 segments

Example: Processing One Run

Let's say you run down a 100m street:

1. **Buffer:** Creates ~2000 sq meter polygon (100m × 20m wide)
2. **Subdivide:** Splits into ~8 smaller polygons
3. **Intersect:** Tests 20 segments (100m ÷ 5m) against 8 polygons = 160 tests
4. **Result:** 18 segments marked visited (90% coverage)
 - Why not 100%? GPS track started 5m from intersection, stopped 5m early

Performance Characteristics

Operation	Time Complexity	Memory Usage
Buffer run	O(track points)	O(1 buffer)
Subdivide	O(vertices)	O(subdivisions)
Intersect	O(segments touched)	O(segments touched)
Aggregate	O(blocks touched)	O(blocks touched)
Total per run	O(streets nearby)	O(streets nearby)

Key insight: Processing time and memory scale with *geography of this run*, not with *total run count*.

Further Reading

- [BLOCKPERCENTAGEMIGRATION.md](#) - Why we use blocks
- [PERFORMANCE_OPTIMIZATION.md](#) - Memory optimization details
- [SEGMENTVISITEDMIGRATION.md](#) - Technical migration guide
- [PostGIS Documentation](#) - Full spatial function reference